MINGGU 6

PEMODELAN SISTEM dengan UML

 Copyright © September 2026

Fokus Pembelajaran

1. Pemodelan Sistem
2. Unified Modeling Language
3. Use Case Diagram
4. Acivity Diagram
5. Sequence Diagram
6. Studi Kasus

Copyright © September 2026

1. PEMODELAN SISTEM

- Model membantu pengguna, manajer, dan profesional IT memahami desain suatu sistem.
- Pemodelan sistem adalah proses pengembangan model abstrak suatu sistem, dimana masing-masing model menyajikan pandangan atau perspektif yang berbeda dari sistem tersebut.
- Pemodelan sistem juga berarti merepresentasikan sistem menggunakan semacam notasi grafis, seperti UML.
- Aspek terpenting dari model sistem adalah tidak adanya detail.
- Model merupakan abstraksi dari sistem yang sedang dipelajari dan bukan representasi alternatif dari sistem tersebut. Representasi sistem harus memelihara semua informasi tentang entitas yang diwakili.

Copyright © September 2026

Pemodelan Sistem (Lanjutan)

Model sistem akan dikembangkan pada:

1. Model **sistem yang ada** digunakan selama rekayasa persyaratan untuk membantu memperjelas apa yang dilakukan sistem yang ada dan dapat digunakan sebagai dasar untuk mendiskusikan kekuatan dan kelemahannya. Ini kemudian mengarah pada persyaratan untuk sistem baru.
2. Model **sistem baru** digunakan selama rekayasa persyaratan untuk membantu menjelaskan persyaratan yang diusulkan kepada Stakeholder.

Copyright © September 2026

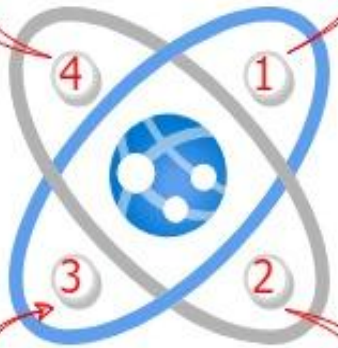
Perspektif dalam mengembangkan model

Perspektif Struktural

memodelkan organisasi suatu sistem atau struktur data yang diproses oleh sistem

Perspektif Ekternal

untuk memodelkan konteks atau lingkungan sistem



Perspektif Perilaku

memodelkan perilaku dinamis sistem dan bagaimana sistem merespons peristiwa

Perspektif Interaksi

untuk memodelkan interaksi antara suatu sistem dan lingkungannya atau diantara komponen-komponen sistem

Copyright © September 2026

2. UNIFIED MODELING LANGUAGE (UML)

- Menurut Booch, et.al. “UML adalah bahasa standar untuk menulis *blue print* PL. UML dapat digunakan untuk memvisualisasikan, menentukan, membuat, dan mendokumentasikan artefak dari sistem PL secara intensif”.
- UML sesuai untuk sistem pemodelan mulai dari sistem informasi perusahaan, aplikasi berbasis web yang terdistribusi, bahkan sampai sistem *real time embedded* yang sulit.
- UML adalah proses yang independen, walaupun secara optimal harus digunakan dalam proses yang menggunakan *case driven*, *architecture-centric*, *iterative*, dan *incremental*.

Copyright © September 2026

UML adalah bahasa untuk:

a. Visualizing

Beberapa hal dimodelkan secara tekstual atau dengan model grafis. UML adalah bahasa grafis yang menggunakan sekelompok simbol grafis. Setiap simbol dalam notasi UML didefinisikan dengan baik secara semantik, sehingga pengembang dapat menulis model UML dan dapat menafsirkan model itu dengan jelas.

b. Specifying

UML dapat membangun model yang tepat, tidak ambigu, dan lengkap. UML membahas spesifikasi semua keputusan analisis, perancangan, dan implementasi penting yang harus dilakukan dalam mengembangkan dan menerapkan sistem PL yang intensif.

Copyright © September 2026

UML adalah bahasa untuk (lanjutan)

c. Constructing

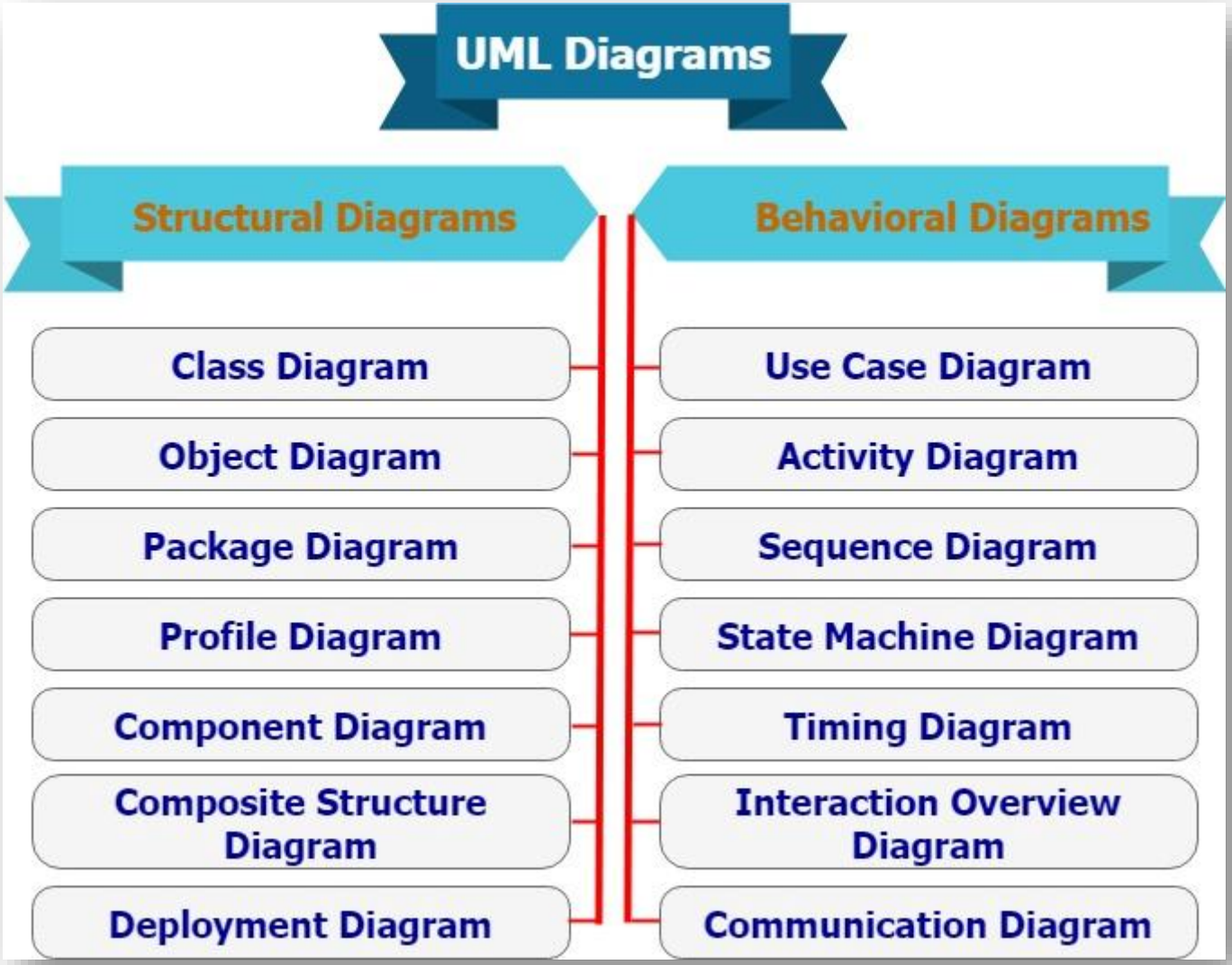
UML bukan bahasa pemrograman visual, namun modelnya bisa langsung terhubung ke berbagai bahasa pemrograman, dan memungkinkan untuk memetakan ke bahasa pemrograman seperti Java, C ++, atau Visual Basic, atau bahkan ke tabel dalam basis data relasional atau penyimpanan database berorientasi objek yang tetap.

d. Documenting

UML membahas dokumentasi arsitektur sistem dan semua detailnya. UML menyediakan bahasa untuk mengekspresikan persyaratan dan tes. UML juga menyediakan bahasa untuk memodelkan kegiatan perencanaan proyek.

Copyright © September 2026

UML versi 2.0 dibagi menjadi 2 kelompok:



 Copyright © September 2026

Diagram-Diagram dalam UML

1. ***Class Diagram***

   Menunjukkan seperangkat kelas, antarmuka, dan kolaborasi dan hubungan di antara mereka. Class Diagram membahas desain statis dari suatu sistem.

2. ***Object Diagram***

   Menunjukkan satu set objek dan hubungan antara objek. Diagram objek memodelkan *instance* dari hal-hal yang terdapat dalam *Class Diagram*. Diagram objek digunakan untuk memodelkan desain statis suatu sistem, untuk memvisualisasikan, menentukan, dan mendokumentasikan model struktural, dan membangun aspek statis sistem melalui teknik maju (*forward*) dan mundur (*reverse*).

Copyright © September 2026

Diagram-Diagram dalam UML (Lanjutan)

3. ***Component Diagram***

   Menunjukkan organisasi dan ketergantungan antar sekumpulan komponen.

4. ***Deployment Diagram***

   Menunjukkan konfigurasi komponen dalam proses eksekusi aplikasi. Diagram ini terdiri dari node yang merupakan perangkat keras dan membungkus satu atau lebih komponen.

5. ***Use Case Diagram***

   Menunjukkan sebuah interaksi antara satu atau lebih aktor dengan sistem informasi yang akan dibuat. Diagram ini sangat penting dalam mengatur dan memodelkan perilaku suatu sistem.

Copyright © September 2026

Diagram-Diagram dalam UML (Lanjutan)

6. ***Sequence Diagram***

   Diagram interaksi yang menekankan urutan waktu pada pesan. Menempatkan objek yang berpartisipasi dalam interaksi pada sumbu X dan menempatkan pesan antar objek sepanjang sumbu Y, sedangkan waktu digambarkan dari atas ke bawah.

7. ***Activity Diagram***

   Diagram yang menunjukkan arus dari aktivitas ke aktivitas dalam suatu sistem. *Activity Diagram* membahas pandangan dinamis suatu sistem, dan sangat penting dalam pemodelan fungsi suatu sistem dan menekankan aliran kontrol antar objek.

Copyright © September 2026

Diagram-Diagram dalam UML (Lanjutan)

8. *Statechart Diagram*

Menggambarkan perubahan status atau transisi status dari sebuah mesin atau sistem atau objek. *Statechart* sangat penting dalam memodelkan perilaku antarmuka, kelas, atau kolaborasi dan menekankan perilaku dari suatu objek, yang sangat berguna dalam memodelkan sistem reaktif.

9. *Collaboration Diagram*

Diagram interaksi yang menekankan struktur organisasi objek yang mengirim dan menerima pesan. Diagram ini merupakan perluasan dari diagram objek, yaitu memberikan tambahan asosiasi antara objek, dan menunjukkan objek mengirimkan *message* ke objek-objek yang lain.

Copyright © September 2026

Diagram-Diagram dalam UML (Lanjutan)

10. *Package Diagram*

Diagram paket sering digunakan untuk melihat ketergantungan antar paket. Karena sebuah paket dapat rusak jika paket lain yang menjadi sandarannya berubah. Paket paket juga biasa digunakan untuk mengelompokkan Use Case dan Class.

11. *Profile Diagram*

Adalah mekanisme ekstensibilitas yang memungkinkan memperluas dan menyesuaikan UML dengan menambahkan blok penyusun baru, membuat properti baru, dan menentukan semantik baru agar bahasanya sesuai dengan domain masalah yang spesifik.

Copyright © September 2026

Diagram-Diagram dalam UML (Lanjutan)

12. Composite Structure *Diagram*

Adalah diagram struktural UML yang berisi kelas, antarmuka, paket, dan hubungannya, dan memberikan pandangan logis dari semua, atau bagian dari sistem perangkat lunak.

13. Timing Diagram

Adalah diagram interaksi yang digunakan untuk menunjukkan interaksi ketika tujuan utama diagram adalah untuk mempertimbangkan waktu.

Copyright © September 2026

Diagram-Diagram dalam UML (Lanjutan)

14. ***Interaction Diagram***

    Diagram UML yang digunakan untuk menangkap perilaku interaktif suatu sistem. Diagram interaksi fokus pada menggambarkan aliran pesan dalam suatu sistem. Diagram ini juga digunakan untuk mewakili urutan yang terurut dalam suatu sistem dan bertindak sebagai sarana memvisualisasikan data real-time melalui UML.

15. ***Communication Diagram***

    Sejenis diagram interaksi, yang menunjukkan bagaimana objek berinteraksi. Diagram komunikasi adalah perpanjangan dari diagram objek yang menunjukkan objek beserta pesan yang berpindah dari satu objek ke objek lainnya.

Copyright © September 2026

2. USE CASE DIAGRAM

- Use Case Diagram digunakan untuk menggambarkan serangkaian tindakan (*use cases*) bahwa sistem dapat melakukan interaksi di luar sistem (aktor) dengan sistem itu sendiri (abstraksi).
- U*se Case* juga digunakan untuk mengetahui fungsi apa saja yang ada dalam sebuah sistem dan siapa yang berhak menggunakan fungsi-fungsi itu.
- Nama Use Case didefinisikan semudah mungkin dan dapat dipahami.
- Dua hal utama pada Use Case yaitu pendefinisian aktor dan use case.

Copyright © September 2026

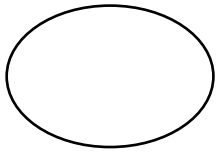
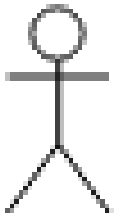
Use Case Diagram (Lanjutan)

Use case digunakan untuk:

- Merepresentasikan interaksi sistem – pengguna
- Mendefinisikan dan mengatur persyaratan fungsional dalam suatu sistem
- Menentukan konteks dan persyaratan sistem
- Memodelkan aliran dasar peristiwa dalam use case

Copyright © September 2026

a. Simbol-Simbol Use Case

No	Nama	Simbol	Keterangan
1.	Use Case		• Digambarkan dengan elips horizontal • Nama Use case menggunakan **kata kerja**
2.	Aktor		• Menggambarkan orang, system/external entitas yang menyediakan atau menerima informasi • Merupakan lingkungan luar dari sistem • Nama Aktor menggunakan **Kata benda** • Aktor **utama** digambarkan pada pojok kiri atas dari diagram

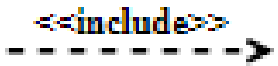
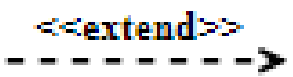
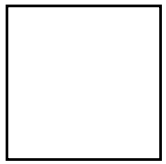
 Copyright © September 2026

Simbol-Simbol Use Case

No	Nama	Simbol	Keterangan
3.	Asosiasi		• Menggambarkan **bagaimana aktor** berinteraksi dengan use case • **Bukan** menggambarkan aliran data/informasi
4.	Generalisasi		• Gambarkan generalisasi antara use case atau antara aktor dengan panah tertutup yang mengarah dari *child* ke *parent*

Copyright © September 2026

Simbol-Simbol Use Case

No	Nama	Simbol	Keterangan
5.	Relasi **include**		• Hubungan antara dua use case untuk menunjukkan adanya perilaku use case yang dimasukkan ke dalam perilaku dari *base use case* • Tanda panah terbuka harus terarah ke ***sub use case***
	Relasi **extend**		• Perluasan dari use case lain (*optional*) • Tanda panah terbuka harus terarah ke ***base use case***
6.	Boundary Boxes		• Untuk memperlihatkan batasan sistem dengan lingkungan luar sistem

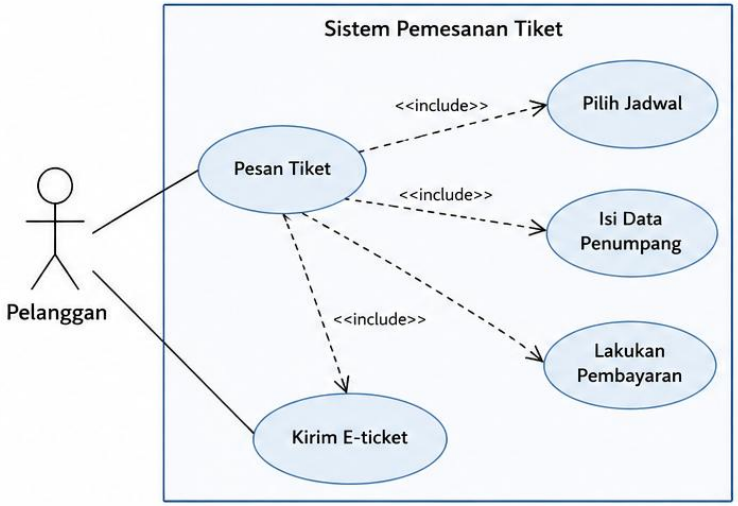
 Copyright © September 2026

Contoh penggunaan relasi *include* dan *extend*

CONTOH PENGGUNAAN <<include>>

<<include>> digunakan ketika suatu use case selalu memerlukan use case lain sebagai bagian dari prosesnya.

Contoh Kasus: Sistem Pemesanan Tiket



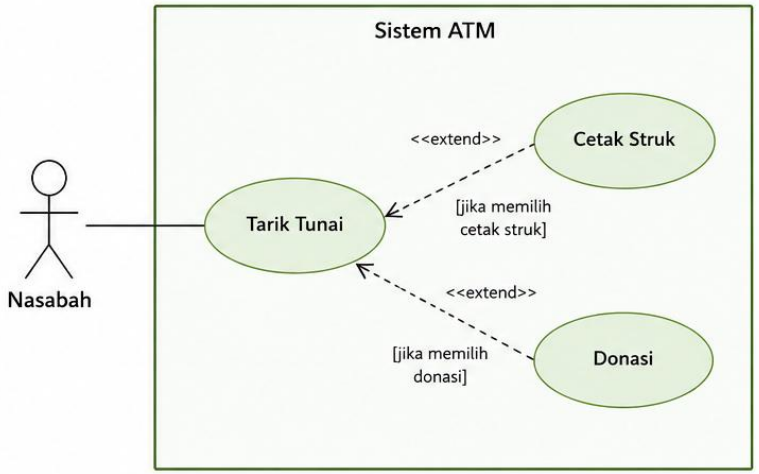
Penjelasan:

- Use case "Pesan Tiket" selalu memerlukan use case: "Pilih Jadwal", "Isi Data Penumpang", "Lakukan Pembayaran", dan "Kirim E-ticket".
- Jika salah satu use case yang di-include berubah, otomatis use case utama juga terpengaruh.

CONTOH PENGGUNAAN <<exclude>> (extend)

<<exclude>> atau <<extend>> digunakan ketika suatu use case merupakan tambahan yang bersifat opsional (hanya terjadi pada kondisi tertentu).

Contoh Kasus: Sistem Penarikan Tunai ATM



Penjelasan:

- Use case "Cetak Struk" dan "Donasi" hanya terjadi jika nasabah memilih opsi tersebut.
- Use case utama "Tarik Tunai" tetap berjalan meskipun use case tambahan tidak dilakukan.
- Perubahan pada use case tambahan tidak mempengaruhi use case utama.

Copyright © September 2026

b. Deskripsi Use Case

- Use Case Diagram merupakan titik awal yang bagus untuk memahami sistem, tetapi tidak memberikan detail yang cukup bagi Analis sistem untuk benar-benar memahami dengan tepat bagaimana masalah sistem akan dipenuhi.
- Cara terbaik untuk mengungkapkan informasi penting ini adalah dalam bentuk deskripsi berbasis teks untuk setiap use case.
- Tidak ada aturan yang jelas dan cepat tentang apa sebenarnya yang masuk ke dalam deskripsi use case menurut UML.

Copyright © September 2026

Deskripsi Use Case

Use Case Name	Nama Use Case yang akan dijelaskan
Requirement	Indikasi tentang persyaratan mana yang dipenuhi oleh use case ini sebagian atau seluruhnya
Goal	Tempat use case di dalam sistem dan mengapa use case ini penting
Pre-Conditions	Apa yang terjadi sebelum use case dapat dieksekusi
Post-Conditions	Apa yang terjadi setelah use case dieksekusi
Failed End Conditions	Bagaimana kondisi sistem jika use case gagal dijalankan dengan sukses
Primary Actor	Aktor utama yang berpartisipasi dalam use case
Main Flow	Tempat untuk mendeskripsikan setiap langkah penting dalam eksekusi normal use case.
Alternatif Flow	Deskripsi langkah-langkah alternatif apa pun dari langkah-langkah yang dijelaskan di Alur Utama.

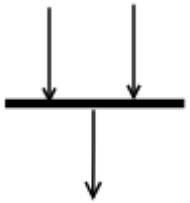
Copyright © September 2026

3. ACTIVITY DIAGRAM

- Activity Diagram adalah teknik untuk menggambarkan logika prosedural, proses bisnis, dan jaringan kerja antara pengguna dan sistem.
- Menggunakan notasi yang **mirip flowchart**, meskipun terdapat sedikit perbedaan notasi karena diagram ini mendukung behavior paralel.
- Activity diagram dibuat berdasarkan **sebuah atau beberapa use case** pada use case diagram
- Memungkinkan melakukan proses untuk memilih urutan dalam melakukannya atau hanya menyebutkan aturan-aturan rangkaian dasar yang harus diikuti, karena proses-proses sering muncul secara paralel.

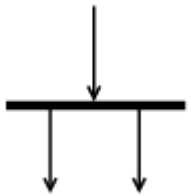
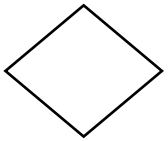
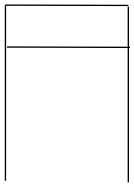
Copyright © September 2026

Simbol-Simbol Activity Diagram

No	Nama	Simbol	Keterangan
1.	Start		• Menjelaskan awal proses kerja dalam activity diagram • Hanya ada satu simbol start
2.	End		• Menandai kondisi akhir dari suatu aktivitas dan merepresentasikan penyelesaian semua arus proses • Bisa lebih dari satu simbol end
3.	Activity		• Menunjukkan kegiatan yang membentuk proses dalam diagram
4.	Join		• Menggabungkan dua atau lebih aktivitas bersamaan dan menghasilkan hanya satu aktivitas yang terjadi dalam satu waktu

 Copyright © September 2026

Simbol-Simbol Activity Diagram

No	Nama	Simbol	Keterangan
5.	Fork		• Membagi aliran aktivitas tunggal menjadi beberapa aktivitas bersamaan
6.	Decision		• Mewakili keputusan yang memiliki setidaknya dua jalur bercabang yang kondisinya sesuai dengan opsi pencabangan
7.	Connector		• Menunjukkan arah aliran atau aliran kontrol dari aktivitas
8.	Swimlane		• Cara untuk mengelompokkan aktivitas berdasarkan aktor • Menggunakan garis vertikal

 Copyright © September 2026

4. SEQUENCE DIAGRAM

- *Sequence diagram* menggambarkan **interaksi antar objek** di dalam dan di sekitar sistem (termasuk pengguna, *display*, dan sebagainya) berupa *message* yang digambarkan terhadap waktu.
- *Sequence diagram* menggambarkan interaksi yang fokusnya pada **urutan pesan** yang dipertukarkan, bersama dengan spesifikasi kemunculannya yang sesuai pada garis hidup.
- *Sequence diagram* terdiri atas **dimensi vertikal** (waktu) dan **dimensi horizontal** (objek-objek yang terkait).
- Diagram ini secara khusus berasosiasi dengan ***use case* diagram**

Copyright © September 2026

Fungsi Sequence Diagram:

a. Menentukan detail dari *Use Case*.

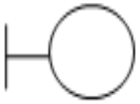
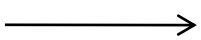
b. Memodelkan logika prosedur, fungsi, atau operasi yang terdapat dalam sistem.

c. Untuk melihat bagaimana objek dan komponen saling berinteraksi satu sama lain untuk menyelesaikan suatu proses.

d. Merencanakan dan memahami fungsionalitas secara rinci dari skenario yang ada atau yang akan datang.

Copyright © September 2026

Simbol Sequence Diagram

No	Simbol	Nama	Fungsi
1	;Objek	Object	Komponen utama Sequence Diagram
2		Actor	Menggambarkan orang yang sedang berinteaksi dengan sistem
3		Entity Class	Menggambarkan hubungan kegiatan yang akan dilakukan
4		Boundary Class	Menggambarkan sebuah penggambaran dari form
5		Control Class	Menggambarkan penghubung antara boundary dengan tabel
6		Life Line	Menggambarkan tempat mulai dan berakhirnya sebuah message
7		Message	Menggambarkan pengiriman pesan

 Copyright © September 2026

Loop dan Kondisi

- *Loop* dan kondisi menggunakan kerangka interaksi, yaitu cara penandaan sebuah bagian *sequence diagram*.
- Kerangka terdiri dari beberapa daerah yang dipisahkan menjadi beberapa ***fragmen***.
- Setiap kerangka memiliki sebuah ***operator***.
- Setiap *fragmen* memiliki sebuah ***guard***.
- *Guard* merupakan sebuah ekspresi kondisional dalam tanda kurung [ ], dan menunjukkan bahwa pesan akan dikirimkan jika nilai *guard* benar.

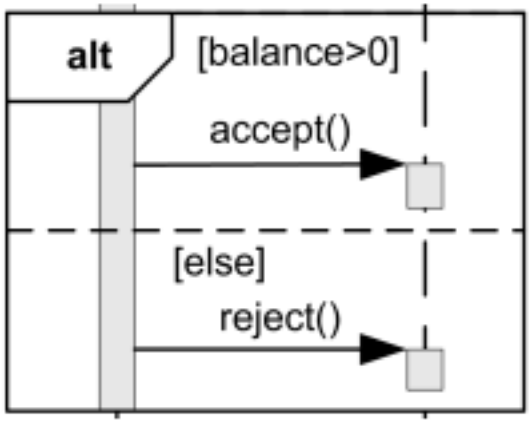
Copyright © September 2026

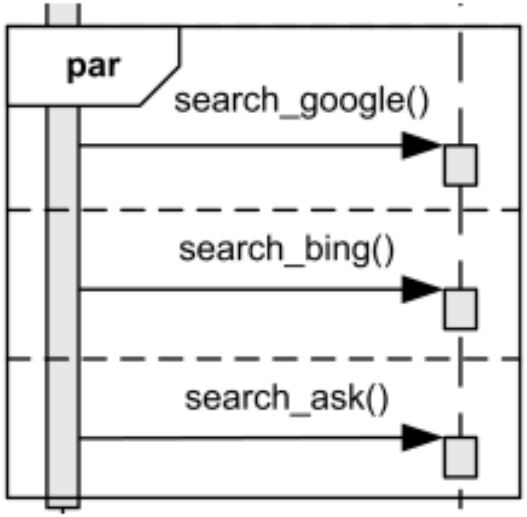
Operator Umum untuk Kerangka Interaksi

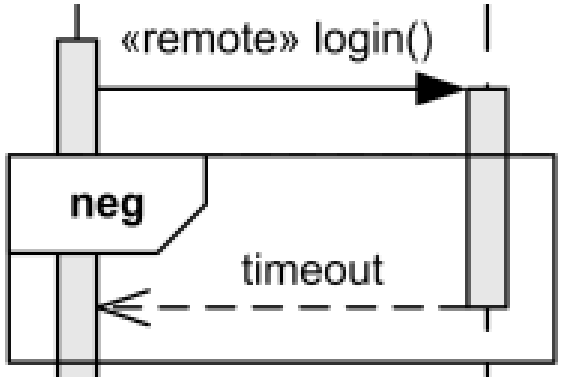
Operator	Keterangan
alt	Alternatif dari banyak fragmen. Hanya yang kondisinya true yang akan dijalankan
opt	Optional; fragmen akan dijalankan jika kondisi yang mendukungnya true
par	Paralel; setiap fragmen dijalankan secara paralel
loop	Looping, fragmen mungkin dijalankan berulang kali dan guard menunjukkan basis iterasi
region	Critical region; fragmen hanya dapat mempunyai satu thread untuk menjalankannya
neg	Negatif; fragmen menunjukkan interaction yang salah
ref	Reference; menunjukkan ke sebuah interaction yang didefinisikan pada diagram yang lain
sd	Sequence diagram

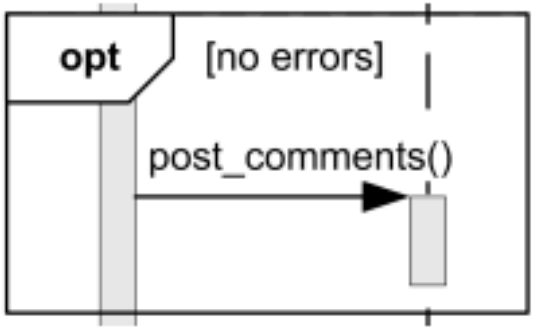
Copyright © September 2026

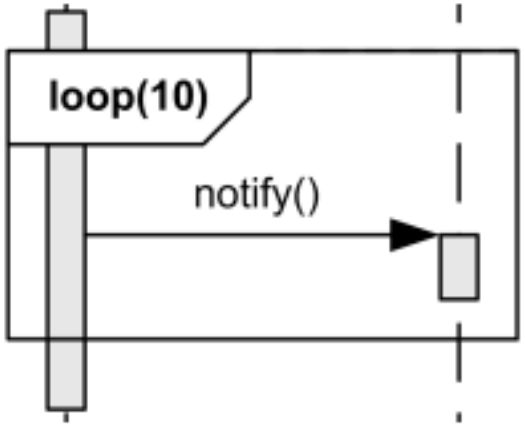
Contoh penggunaan operator:











 Copyright © September 2026

5. CONTOH STUDI KASUS

Tema: Sistem Pemesanan Makanan (Kue) pada Usaha UMKM

Untuk memperluas pemasaran usaha makanan, seorang anggota baru UMKM akan mengembangkan PL menjadi aplikasi berbasis website.

Aplikasi dapat diakses langsung oleh Pelanggan untuk pembelian atau pemesanan produk makanan.

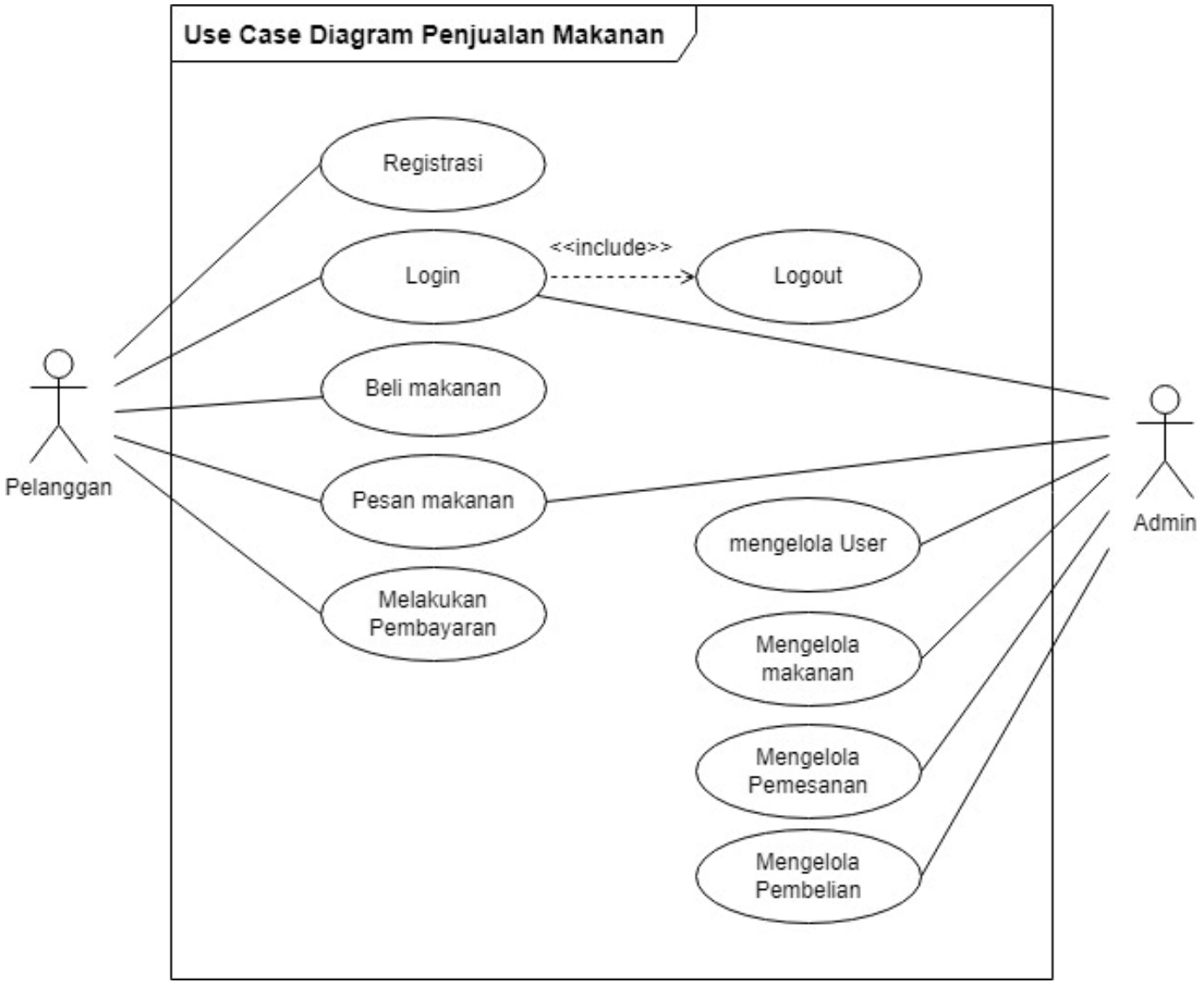
Pelanggan melihat makanan dari katalog produk kemudian dapat langsung membeli atau memesan jika produk tidak tersedia, dengan mengisi form pemesanan.

Pelanggan yang akan membeli/memesan melakukan registrasi dahulu dengan mengisi data berupa nama, email, nomor telp, alamat, dan passwd.

Pembayaran dapat berupa transfer bank, virtual akun, dan COD. Pelanggan yang sudah membayar akan mendapatkan notifikasi bahwa pembayaran sudah diterima.

Copyright © September 2026

A. USE CASE DIAGRAM



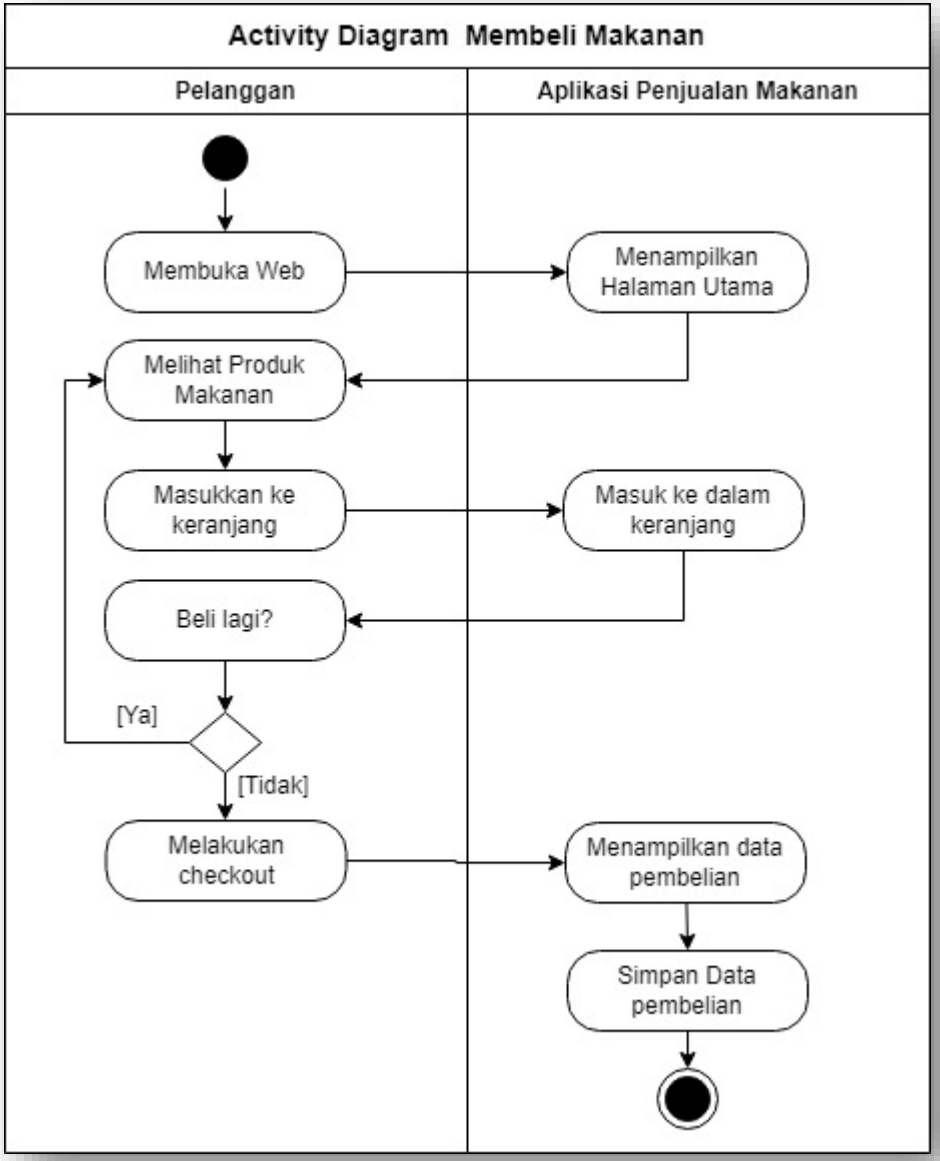
 Copyright © September 2026

DESKRIPSI USE CASE

Use Case Name	**Beli Makanan**
Requirement	Pelanggan memilih produk makanan yang tersedia kemudian memasukkan ke dalam keranjang
Goal	Pelanggan dapat membeli produk makanan yang sudah dimasukkan dalam keranjang
Pre-Conditions	Melihat produk makanan
Post-Conditions	Checkout
Failed End Conditions	Stok makanan tidak tersedia atau habis
Primary Actor	Pelanggan
Main Flow	1. Pelanggan membuka web 2. Pelanggan melihat produk makanan 3. Pelanggan memasukkan makanan ke keranjang 4. Sistem menyimpan data yang ada di keranjang 5. Pelanggan melakukan checkout
Alternatif Flow	-

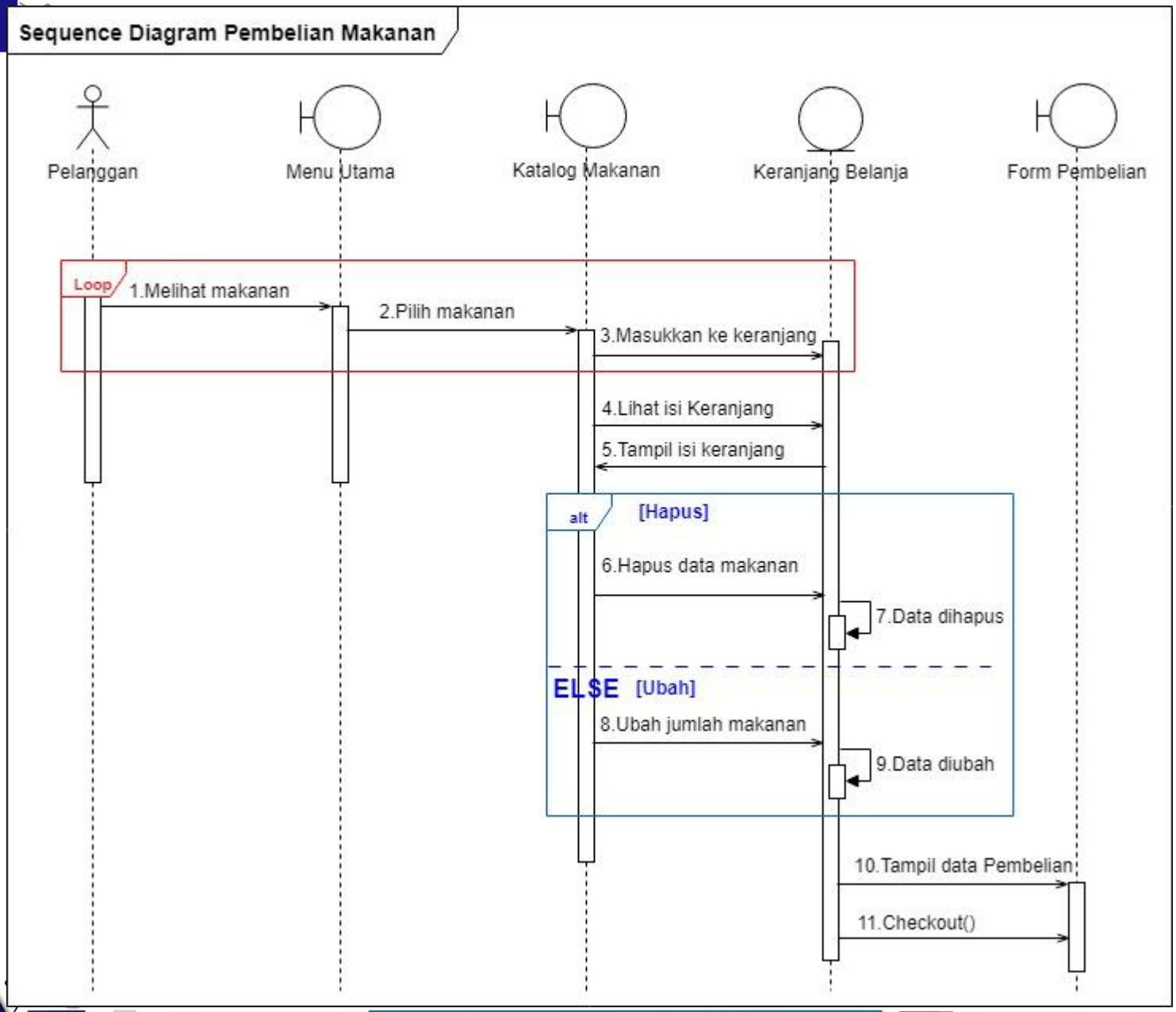
Copyright © September 2026

B. ACTIVITY DIAGRAM



 Copyright © September 2026

C. SEQUENCE DIAGRAM



Copyright © September 2026

Latihan

Dosen memberikan contoh kasus, kemudian memodelkan kasus tersebut dengan diagram-diagram yang sudah dibahas

Copyright © September 2026

TUGAS KELOMPOK

Buatlah diagram sesuai dengan judul tugas pada Minggu ke 2.

Diagram yang dibuat:

- Use Case Diagram
- Activity Diagram
- Sequence Diagram

Copyright © September 2026